

Introduction aux Logiciels Libres

Session 1

Cours M1 – Université Paris 7

Présentation du cours

Description du cours

Ce cours vise à enseigner les aspects **économiques, juridiques et organisationnels** de l'industrie du logiciel libre.

Format

- 12 séances de 3h
- Cours magistraux participatifs
- Evaluation?

Approche

- Moins axé sur la technologie
- Focus sur le fonctionnement de l'industrie
- Mise en pratique des connaissances

Objectifs pédagogiques

À l'issue de ce cours, vous serez capables de :

- **Comprendre** le fonctionnement des projets open source
- **Maîtriser** les aspects juridiques (licences, propriété intellectuelle)
- **Analyser** les modèles économiques du logiciel libre
- **Contribuer** efficacement à des projets open source
- **Appliquer** ces connaissances en entreprise ou en startup

Pourquoi s'en préoccuper ?

En 25 ans, le logiciel libre a radicalement changé la façon dont les logiciels sont :

- conçus et développés
- testés et déployés
- vendus et maintenus
- enseignés

Relations avec :

- Le cloud et les conteneurs
- L'innovation ouverte
- La souveraineté numérique
- L'économie de l'immatériel

Un marché de plusieurs milliards d'euros, >10% du marché logiciel.

Qui je suis

Stefane Fermigier – Entrepreneur et activiste du logiciel libre

Parcours

- Découverte du libre en 1991 (Linux 0.11)
- Co-fondateur de l'AFUL (1998)
- Fondateur de Nuxeo (2000-2012)
- Fondateur d'Abilian (2012-)
- Ex-enseignant-chercheur (maths, Paris 7)

Engagements actuels

- Co-président du CNLL
- Membre du bureau exécutif de l'APELL
- Co-fondateur et membre du CA d'EuroStack
- Promotion du numérique ouvert en France et en Europe

Contact : sf@fermigier.com

Plan du cours

Sessions 1 & 2

Session 1 – Introduction

- Définitions : logiciel libre, open source
- Les 4 libertés fondamentales (FSF)
- Free Software vs Open Source
- L'Open Source Definition (OSI)
- Panorama de l'open source aujourd'hui

Session 2 – Histoire (1950-aujourd'hui)

- Unix et la culture hacker du MIT
- Richard Stallman, GNU, FSF, GPL
- Linux et les distributions
- The Cathedral and the Bazaar, naissance de l'OSI
- Git, GitHub, cloud, "Microsoft loves Linux"

Sessions 3 & 4

Session 3 – Droit et Propriété Intellectuelle

- Droit d'auteur (copyright)
- Droits moraux vs patrimoniaux
- Brevets logiciels (US vs Europe)
- Les marques dans l'open source
- Le copyleft : mécanisme juridique

Session 4 – Licences : Fondamentaux

- Anatomie d'une licence
- Licences permissives : MIT, BSD, Apache
- Licences copyleft : GPL, AGPL, LGPL, MPL
- Copyleft fort vs faible
- Creative Commons, domaine public

Sessions 5 & 6

Session 5 – Licences : Approfondissement

- Compatibilité des licences
- Choisir une licence (choosealicense.com)
- SPDX : identifiants standards
- DCO, CLA, Copyright Assignment
- Dual licensing, changements de licence

Session 6 – Contribuer : Fondamentaux

- Pourquoi contribuer ?
- Évaluer la santé d'un projet
- Types de contributions (pas que du code)
- Structure d'un projet (README, CONTRIBUTING...)
- Workflow : Fork → Branch → PR → Merge

Sessions 7 & 8

Session 7 – Contribuer : Bonnes pratiques

- Anatomie d'une bonne PR
- Messages de commit (Conventional Commits)
- Communication asynchrone
- Recevoir et donner une code review
- Signed-off-by, DCO, rebase

Session 8 – Gouvernance

- Le droit au fork
- Modèles : BDFL, méritocratie Apache, comités
- Prise de décision (lazy consensus, RFC/PEP)
- Devenir mainteneur
- Les fondations (Apache, Linux Foundation...)

Sessions 9 & 10

Session 9 – Modèles économiques

- Le paradoxe de l'open source
- Dual licensing, Open core
- Support / Distributeur (Red Hat)
- SaaS et le "cloud problem"
- Tensions communauté / business

Session 10 – Supply Chain et Sécurité

- La supply chain logicielle
- Incidents : Log4Shell, XZ Utils
- SBOM (Software Bill of Materials)
- Gestion des vulnérabilités (CVE, CVSS)
- Outils : Dependabot, Snyk, OpenSSF Scorecard

Sessions 11 & 12

Session 11 – Open Source en Entreprise

- L'OSPO (Open Source Program Office)
- InnerSource : l'open source en interne
- Conformité licences
- Cyber Resilience Act (EU)

Session 12 – IA et Open Source + Synthèse

- Définir l'IA "open source"
- Acteurs : LLaMA, Mistral, Hugging Face
- Licences IA et enjeux spécifiques
- Synthèse et bilan du cours
- Ressources pour continuer

Session 1

Objectifs de la séance

Ce que vous saurez faire

À la fin de cette séance, vous serez capables de :

- **Définir** les concepts de "logiciel libre" et "open source"
- **Expliquer** les 4 libertés fondamentales
- **Identifier** les logiciels libres dans votre environnement quotidien
- **Distinguer** les nuances philosophiques entre "free software" et "open source"

Partie 1

Qu'est-ce qu'un logiciel libre ?

Un logiciel, c'est quoi ?

Code source

- Instructions lisibles par un humain
- Écrit dans un langage de programmation
- Peut être étudié et modifié

Code binaire (exécutable)

- Instructions pour la machine
- Résultat de la compilation
- Illisible (ou presque) pour un humain (et pour un LLM?)

Sans le code source, impossible de comprendre ou modifier un logiciel.

Logiciel propriétaire vs Logiciel libre

Logiciel propriétaire

- Code source fermé (secret)
- Utilisation sous conditions (licence)
- Interdiction de copier, modifier
- Exemples : Windows, Photoshop, Office

Logiciel libre

- Code source ouvert (accessible)
- Libertés garanties à l'utilisateur
- Droit de copier, modifier, redistribuer
- Exemples : Linux, Firefox, LibreOffice

Les 4 libertés fondamentales

Définition de la Free Software Foundation (FSF) :

Liberté	Description
Liberté 0	Utiliser le programme, pour n'importe quel usage
Liberté 1	Étudier le fonctionnement du programme (accès au code source)
Liberté 2	Redistribuer des copies
Liberté 3	Modifier le programme et redistribuer les versions modifiées

Ces 4 libertés sont indissociables. Si l'une manque, le logiciel n'est pas libre.

"Free as in freedom, not as in free beer"

Le mot "free" en anglais est ambigu :

- **Free** = gratuit (free beer) X
- **Free** = libre (free speech) V

Un logiciel libre peut être vendu

- La liberté n'interdit pas le commerce
- On peut vendre des copies
- On peut vendre du support

Un logiciel gratuit n'est pas forcément libre

- Freeware ≠ Free Software
- Exemple : Chrome est gratuit mais pas libre (mais Chromium est libre)
- Le code source reste propriétaire (au moins en partie)

Contre-exemples

Qu'est-ce qui **n'est pas** un logiciel libre ?

Type	Exemple	Problème
Propriétaire classique	Microsoft Office	Code fermé, licence restrictive
Freeware	WhatsApp	Gratuit mais code fermé
Shareware	WinRAR	Code fermé, limitation temporelle
Source available	Elastic (SSPL)	Code visible mais licence non-libre
Open core	GitLab EE	Partie libre (CE), partie propriétaire (EE)

Partie 2

Free Software vs Open Source

Deux termes, deux philosophies

Free Software (1983)

- Fondé par Richard Stallman
- Accent sur l'**éthique** et la liberté
- "Les utilisateurs méritent la liberté"
- Mouvement social et politique

Open Source (1998)

- Fondé par Eric Raymond, Bruce Perens
- Accent sur les **avantages pratiques**
- "Le code ouvert produit de meilleurs logiciels"
- Approche pragmatique et business-friendly

"Open source is a development methodology; free software is a social movement." – Richard Stallman

Les définitions formelles

Free Software Definition (FSF)

- Les 4 libertés
- Focus sur l'utilisateur
- Perspective éthique

Open Source Definition (OSI)

- 10 critères
- Focus sur la distribution
- Perspective pratique

En pratique : Les deux définitions sont quasi-équivalentes. Dans la quasi-totalité des cas, un logiciel libre est open source, et vice versa.

On utilise parfois : **FOSS** (Free and Open Source Software) ou **FLOSS** (Free/Libre and Open Source Software)

L'Open Source Definition (OSD)

Les 10 critères de l'Open Source Initiative (1998) :

#	Critère	Description
1	Libre redistribution	Pas de redevance sur la vente/distribution
2	Code source	Doit être disponible ou facilement accessible
3	Œuvres dérivées	Modifications et redistribution autorisées
4	Intégrité du code source	Peut exiger des patches séparés
5	Pas de discrimination (personnes)	Ouvert à tous
6	Pas de discrimination (domaines)	Tout usage autorisé (commerce, recherche...)
7	Distribution de licence	S'applique sans accord supplémentaire
8	Non spécifique à un produit	Droits indépendants du contexte
9	Non restrictive	N'impose pas de contraintes sur autres logiciels
10	Neutre technologiquement	Pas de dépendance à une technologie

Pourquoi cette distinction importe ?

Quand ça compte :

- Discussions philosophiques
- Choix de terminologie officielle
- Positionnement politique
- Communication d'entreprise

Quand ça ne compte pas :

- Choix technique d'un logiciel
- Licence d'un projet
- Contribution à un projet
- Usage quotidien

Pour ce cours : nous utiliserons les deux termes de manière interchangeable, sauf mention contraire.

Partie 3

L'open source aujourd'hui

L'open source est partout

Domaine	Exemples de logiciels libres
Systèmes d'exploitation	Linux, Android, FreeBSD
Serveurs web	Apache, Nginx
Bases de données	PostgreSQL, MySQL, MariaDB, SQLite
Langages	Python, Rust, Go
Développement	Git, Vim/Neovim, Eclipse, Docker
Navigateurs	Firefox, Chromium
Bureautique	LibreOffice, GIMP, Inkscape
Cloud	Kubernetes, OpenStack
IA/ML	TensorFlow, PyTorch, Hugging Face

Quelques chiffres

- **96%** des base de code contiennent de l'open source (Synopsys 2023)
- **90%** des entreprises utilisent de l'open source (Red Hat 2023)
- **100%** des supercalculateurs ("TOP500") et 60% des smartphone utilisent Linux
 - Mais seulement 4% des desktops (en France)
- **GitHub** : 100+ millions de développeurs, 400+ millions de dépôts
 - Attention à ne pas confondre "GitHub" et "Open Source" cependant

L'open source n'est plus une alternative marginale, c'est le standard de l'industrie.

Qui développe l'open source ?

Contributeurs individuels

- Bénévoles passionnés
- Étudiants
- Développeurs freelance

Organisations

- Entreprises (Google, Microsoft, Red Hat...)
- Fondations (Apache, Linux Foundation...)
- Universités et recherche

Chiffre clé : Sur le noyau Linux, plus de **80%** des contributions viennent de développeurs payés par des entreprises.

Pourquoi contribuer à l'open source ?

Pour les individus

- Apprentissage et compétences
- Réputation et visibilité
- Portfolio de code public
- Réseau professionnel
- Plaisir et passion

Pour les entreprises

- Réduction des coûts
- Innovation partagée
- Recrutement de talents
- Influence sur les standards
- Éviter le vendor lock-in

Résumé

Ce qu'il faut retenir

- 1 Un **logiciel libre** garantit 4 libertés : utiliser, étudier, redistribuer, modifier
- 2 "Free" signifie **libre**, pas gratuit
- 3 **Free Software** et **Open Source** désignent la même réalité technique, avec des philosophies différentes (complémentaires?)
- 4 L'open source est **omniprésent** dans l'industrie du logiciel
- 5 Tout le monde peut **utiliser** et **contribuer** à l'open source

Pour la prochaine séance

Séance 2 : Histoire du logiciel libre (1950-aujourd'hui)

Préparation suggérée :

- Lire le GNU Manifesto : <https://www.gnu.org/gnu/manifesto.html>
- Explorer votre système : quels logiciels libres utilisez-vous ?

Questions ?

Passons au TD